

Ejercicio de Aula: Patrón Decorator

Por:

Jairo Yeison Sánchez López

Docente:

Christian David Jaimes Acevedo

Facultad de Ingenierías

Ingeniería de Sistemas

Tecnológico de Antioquia - Institución Universitaria

Los Patios, Colombia

2025

Contenido

INTRODUCCION	3
COMPARACIÓN DECORATOR VS HERENCIA.....	4
Código 1: Interfaz Enemigo	4
Código 2: Decorador base MejoraEnemigo	5
Código 3: Un enemigo base (Slayer)	6
Código 4: Un ejemplo de Decorador (BuffAtaque)	6
Código 5: Combinación dinámica en el Main.....	8
Conclusión.....	9

INTRODUCCION

En el desarrollo de un videojuego con enemigos personalizables, se necesita un sistema flexible que permita agregar equipamientos como armadura, casco, escudo o aumento de ataque.

Usar herencia múltiple o crear clases como `EnemigoConArmaduraYCascoYBuff` genera un código difícil de mantener, rígido y lleno de combinaciones infinitas.

El Patrón Decorator permite agregar funcionalidades de forma dinámica, sin modificar clases existentes y sin crear subclases innecesarias.

COMPARACIÓN DECORATOR VS HERENCIA

Herencia múltiple (mala idea):

- Obliga a crear muchas clases.
- No permite combinar equipamientos en tiempo de ejecución.
- Si agregas un nuevo tipo de mejora, debes modificar muchas clases.

Patrón Decorator (solución flexible):

- Permite agregar mejoras a un enemigo de forma dinámica.
- No requiere modificar código existente.
- Cada mejora es independiente (casco, armadura, escudo, buff de ataque).
- Se pueden combinar en cualquier orden.

Por eso el Decorator es superior: permite extender comportamiento **sin heredar**, solo envolviendo objetos.

Código 1: Interfaz Enemigo

```
package juegopatrondecorator;

public interface Enemigo {
    String getInfo();
    int getAtaque();
    int getDefensa();
}
```

Esta interfaz define el comportamiento base de cualquier enemigo. El patrón Decorator funciona gracias a que todos los enemigos y sus mejoras comparten la misma interfaz.

Código 2: Decorador base MejoraEnemigo

```
package juegopatrondecorator;

public abstract class MejoraEnemigo implements Enemigo {

    protected Enemigo enemigo;

    public MejoraEnemigo(Enemigo enemigo) {
        this.enemigo = enemigo;
    }

    @Override
    public String getInfo() {
        return enemigo.getInfo();
    }

    @Override
    public int getAtaque() {
        return enemigo.getAtaque();
    }

    @Override
    public int getDefensa() {
        return enemigo.getDefensa();
    }
}
```

Esta **clase** **es** **el** **Decorator** **base.** Contiene un objeto Enemigo que envuelve, y delega todas las funciones a él. Las subclases (casco, armadura, buff, escudo) extenderán esta clase para modificar solo lo necesario.

Código 3: Un enemigo base (Slayer)

```
package juegopatrondecorator;

public class Slayer implements Enemigo {

    @Override
    public String getInfo() {
        return "Slayer ";
    }

    @Override
    public int getAtaque() {
        return 1700;
    }

    @Override
    public int getDefensa() {
        return 1000;
    }
}
```

Un enemigo básico que implementa la interfaz. Los decoradores agregan mejoras a este enemigo sin alterarlo.

Código 4: Un ejemplo de Decorador (BuffAtaque)

```
1 package juegopatrondecorator;
2
3 public class BuffAtaque extends MejoraEnemigo {
4
5     public BuffAtaque(Enemigo enemigo) {
6         super(enemigo);
7     }
8
9     @Override
10    public String getInfo() {
11        return enemigo.getInfo() + "+ Buff de Ataque ";
12    }
13
14    @Override
15    public int getAtaque() {
16        return enemigo.getAtaque() + 500;
17    }
18 }
19
```

El decorador modifica solo el ataque. Es flexible, reutilizable y se puede combinar con otros decoradores sin necesidad de crear nuevas clases derivadas.

Código 5: Combinación dinámica en el Main

```
package juegopatrondecorator;

public class Main {

    public static void main(String[] args) {

        Enemigo enemigo = new Slayer();

        enemigo = new ProteccionPesada(enemigo);
        enemigo = new CascoLigero(enemigo);
        enemigo = new BuffAtaque(enemigo);

        System.out.println(enemigo.getInfo());
        System.out.println("Final Damage: " + enemigo.getAtaque());
        System.out.println("Final Defense: " + enemigo.getDefensa());
    }
}
```

Aquí se demuestra la potencia del Decorator:

- Se inicia con un enemigo base (Slayer)
- Se le agrega armadura
- Luego casco
- Luego buff de ataque

Todo en tiempo de ejecución, sin crear nuevas clases ni modificar código heredado.

Conclusión

El patrón Decorator permite extender el comportamiento de un enemigo de forma flexible, dinámica y ordenada. En lugar de usar herencias múltiples o crear clases gigantes con todas las combinaciones posibles, Decorator permite agregar equipamientos como módulos independientes que se pueden apilar libremente.

Esto hace que el código sea más escalable, limpio y fácil de mantener.